

SmartFlow IT

Conception et développement d'une plateforme intelligente de gestion des demandes internes et de pilotage des SLA

CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

Stagiaire	Hamza Bouchenou
Formation	Cycle ingénieur - Management et Gouvernance des Systèmes d'Information
Établissement	ENSA Khouribga
Entreprise d'accueil	Neonovia
Encadrant	Youssef OUSSOUK
Période prévisionnelle	3 mois

Fiche de suivi du document

Élément	Valeur
Titre du document	Cahier des charges fonctionnel et technique - SmartFlow IT
Nature	Sujet de projet de fin d'études
Version	1.0
Statut	Document finalisé pour lancement du projet
Stagiaire	Hamza Bouchenou
Durée cible	12 semaines

Historique des versions

Version	Date	Auteur	Description
1.0	03/08/2026	Youssef OUSSOUK	Finalisation du cahier des charges

Sommaire

1. Présentation du sujet
2. Contexte et problématique
3. Objectifs du projet
4. Périmètre fonctionnel
5. Acteurs et droits d'accès
6. Besoins fonctionnels détaillés
7. Règles de gestion
8. Exigences non fonctionnelles
9. Conception générale
10. Modèle de données
11. Interfaces et API
12. Module d'intelligence artificielle
13. Sécurité et traçabilité
14. Stratégie de tests
15. Déploiement et DevOps
16. Planning prévisionnel
17. Livrables et critères de recette
18. Risques et points de vigilance
19. Organisation du suivi
20. Conclusion

1. Présentation du sujet

Le projet SmartFlow IT consiste à concevoir et développer une application web permettant de centraliser les demandes internes d'une organisation, de les faire circuler dans des circuits de validation configurables et de suivre leur traitement à l'aide d'indicateurs de service. La solution doit couvrir tout le cycle de vie d'une demande : création, qualification, validation, affectation, traitement, contrôle et clôture.

Le sujet associe le développement full-stack, la modélisation des processus, la gouvernance des accès, la mesure des SLA et l'exploitation raisonnée de l'intelligence artificielle. Le périmètre retenu est réalisable sur trois mois, avec une organisation du travail par lots et une priorité donnée au socle fonctionnel.

Intitulé du projet - Conception et développement d'une plateforme intelligente de gestion des demandes internes et de pilotage des engagements de service (SLA).

1.1. Pourquoi ce sujet

Dans beaucoup d'organisations, les demandes internes sont encore transmises par e-mail, feuilles Excel ou messages instantanés. Cette organisation rend le suivi difficile : responsabilités peu claires, délais non maîtrisés, relances manuelles, historique incomplet et absence de données fiables pour le pilotage.

SmartFlow IT répond à ce besoin par une application unique, avec des circuits transparents et une traçabilité complète. Le sujet est également adapté à un profil maîtrisant Spring Boot, React, TypeScript, PostgreSQL, Python, Docker et Microsoft Azure. Il permet de valoriser à la fois les compétences techniques et les notions de management et gouvernance des systèmes d'information.

1.2. Exemple de cas d'usage

- Demande de matériel informatique ou d'accès à une application.
- Demande d'intervention technique ou de support interne.
- Demande d'achat nécessitant une validation hiérarchique.
- Demande de document administratif ou de prestation interne.
- Déclaration d'un incident avec niveau de priorité et délai de prise en charge.

2. Contexte et problématique

2.1. Situation actuelle retenue pour le projet

L'entreprise d'accueil dispose de plusieurs services internes. Chaque service reçoit des demandes par des canaux différents. Les collaborateurs ne savent pas toujours à qui adresser leur besoin, ni quel délai attendre. Les responsables ne disposent pas d'une vue globale sur la charge, les retards ou les catégories de demandes les plus fréquentes.

Le projet partira de ce contexte générique. Lors de la phase de cadrage, deux ou trois processus réels de l'entreprise pourront être retenus comme pilotes. Le produit sera conçu de manière suffisamment générique pour permettre l'ajout de nouveaux services sans modifier le code métier principal.

2.2. Problématique

Question centrale - Comment mettre en place une plateforme sécurisée et configurable qui simplifie le dépôt des demandes, automatise les validations, mesure les délais de traitement et aide les équipes à prendre de meilleures décisions ?

2.3. Enjeux

Enjeu	Constat	Réponse attendue
Centralisation	Demandes dispersées dans plusieurs canaux	Un portail unique et un catalogue de services
Traçabilité	Difficulté à retrouver les décisions et échanges	Historique horodaté et journal d'audit
Délais	Relances manuelles et absence d'engagement clair	SLA, alertes, escalades et tableaux de bord
Gouvernance	Droits et responsabilités parfois imprécis	Rôles, périmètres et séparation des responsabilités
Pilotage	Indicateurs produits manuellement	KPI calculés à partir des données de traitement
Assistance	Temps perdu à classer et reformuler les demandes	Aide IA pour la classification, le résumé et la recherche documentaire

3. Objectifs du projet

3.1. Objectif général

Livrer une première version opérationnelle de SmartFlow IT, déployable dans un environnement de démonstration ou de recette, avec au moins deux processus pilotes entièrement configurés et testés.

3.2. Objectifs fonctionnels

- Mettre à disposition un catalogue clair des demandes disponibles.
- Permettre la saisie de formulaires adaptés à chaque type de demande.
- Faire circuler les demandes dans un workflow de validation et de traitement.
- Gérer l'affectation, les priorités, les commentaires et les pièces jointes.
- Mesurer les délais de prise en charge et de résolution.
- Notifier les utilisateurs lors des étapes importantes.
- Fournir des tableaux de bord pour les responsables et administrateurs.
- Conserver un historique complet des actions réalisées.

3.3. Objectifs techniques et pédagogiques

- Mettre en œuvre une architecture REST séparant clairement le front-end et le back-end.
- Appliquer des règles de sécurité : authentification, autorisations, validation des entrées et audit.
- Industrialiser le projet avec Git, Docker, tests automatiques et pipeline CI/CD.
- Intégrer un composant Python pour les fonctions IA sans rendre le cœur du produit dépendant du modèle.
- Documenter les choix d'architecture, les API, le modèle de données et la procédure de déploiement.

3.4. Indicateurs de réussite

Indicateur	Cible minimale
Processus pilotes	2 processus configurés de bout en bout
Scénarios de recette	Au moins 25 scénarios fonctionnels exécutés
Couverture des services critiques	Tests unitaires sur règles métier et sécurité
Performance	Affichage des écrans courants en moins de 3 secondes en environnement de recette
Traçabilité	100 % des changements d'état enregistrés dans l'historique
Déploiement	Application démarrable par Docker Compose et version déployée sur Azure ou serveur de recette

4. Périmètre fonctionnel

4.1. Périmètre inclus dans la version PFA

- Authentification et gestion du profil utilisateur.

- Organisation : directions, services, équipes et responsables.
- Catalogue des services et types de demandes.
- Formulaire configurables à partir d'un ensemble de types de champs prédéfinis.
- Création, sauvegarde en brouillon, soumission et consultation des demandes.
- Workflow avec étapes, décisions, retours et commentaires.
- Affectation manuelle ou automatique à un agent ou une équipe.
- Gestion des priorités, échéances, SLA et escalades.
- Notifications dans l'application et par e-mail.
- Tableaux de bord et export CSV des principales listes.
- Administration, journal d'audit et paramétrage général.
- Module IA limité aux fonctions définies au chapitre 12.

4.2. Éléments hors périmètre initial

- Application mobile native iOS/Android. L'interface web devra toutefois être responsive.
- Moteur BPMN complet comparable à une suite BPM du marché.
- Signature électronique qualifiée ou paiement en ligne.
- Connexion à un ERP réel sans accès fourni par l'entreprise.
- Prise de décision automatique par l'intelligence artificielle.
- Haute disponibilité multi-région et exploitation 24 h/24.

Principe de cadrage - Le cœur du PFA doit être terminé avant les fonctions avancées. Le workflow, la sécurité, les SLA et la traçabilité sont prioritaires. Les fonctions IA seront ajoutées après stabilisation du socle.

5. Acteurs et droits d'accès

Acteur	Responsabilités principales	Accès attendu
Collaborateur / demandeur	Créer et suivre ses demandes, répondre aux demandes de complément	Ses demandes et les services qui lui sont ouverts
Manager / valideur	Accepter, rejeter ou retourner une demande	Demandes relevant de son périmètre de validation
Agent de traitement	Qualifier, prendre en charge, commenter et résoudre	File de travail de son équipe et demandes affectées
Responsable de service	Piloter la charge, les délais et les règles de son service	Toutes les demandes et indicateurs de son service
Administrateur fonctionnel	Configurer catalogue, formulaires, workflows, SLA et notifications	Paramétrage fonctionnel global
Administrateur technique	Gérer les comptes techniques, logs, sauvegardes et déploiements	Configuration technique et supervision
Auditeur / direction	Consulter des indicateurs et traces sans modifier les données	Lecture seule sur périmètre autorisé

5.1. Principes d'autorisation

- Les droits seront attribués par rôle et, lorsque nécessaire, limités à un service ou une direction.
- Un utilisateur ne doit pas valider sa propre demande lorsque la règle de séparation des tâches est activée.
- L'accès à une pièce jointe doit respecter les mêmes règles que l'accès à la demande concernée.
- Les opérations d'administration et les changements de droits seront journalisés.

6. Besoins fonctionnels détaillés

6.1. Authentification et profils

- Connexion par identifiant et mot de passe ; intégration Azure AD/Entra ID possible en option.

- Réinitialisation du mot de passe et désactivation d'un compte.
- Consultation et mise à jour des informations de profil autorisées.
- Gestion des rôles, du service de rattachement et du responsable hiérarchique.
- Expiration de session et déconnexion sécurisée.

6.2. Catalogue de services

- Présentation des services sous forme de catégories et de fiches compréhensibles.
- Recherche par mot-clé, catégorie ou service responsable.
- Affichage des informations utiles avant la saisie : description, délai cible, pièces nécessaires et contacts.
- Activation, désactivation et ordre d'affichage des types de demande.
- Visibilité d'un service selon le rôle, l'entité ou le groupe de l'utilisateur.

6.3. Formulaires configurables

- Création d'un formulaire par type de demande à partir de champs texte, nombre, date, liste, case à cocher, utilisateur, service et fichier.
- Gestion du caractère obligatoire, de l'ordre, de l'aide à la saisie et des valeurs possibles.
- Contrôles de validation côté interface et côté serveur.
- Possibilité d'afficher un champ selon la valeur d'un autre champ pour quelques cas simples.
- Conservation des valeurs saisies lorsque la demande est sauvegardée en brouillon.

6.4. Gestion des demandes

- Création, modification du brouillon, soumission et annulation avant prise en charge.
- Génération d'une référence unique lisible, par exemple DEM-2026-000123.
- Ajout de commentaires, mentions et pièces jointes autorisées.
- Affichage d'une frise d'avancement et de l'historique complet.
- Demande d'informations complémentaires avec retour au demandeur.
- Clôture avec motif, solution apportée et niveau de satisfaction facultatif.

6.5. Workflow et validations

- Définition d'étapes ordonnées avec un rôle ou une équipe responsable.
- Actions possibles selon l'étape : valider, rejeter, retourner, affecter, demander un complément ou clôturer.
- Conditions simples basées sur la catégorie, la priorité, le service ou une valeur du formulaire.
- Enregistrement de la décision, de l'auteur, de la date et d'un commentaire éventuel.
- Possibilité pour l'administrateur de publier une nouvelle version du workflow sans modifier les demandes déjà en cours.

6.6. Affectation et files de travail

- File personnelle « Mes tâches » et file d'équipe.
- Affectation manuelle à un agent habilité.
- Affectation automatique selon le service, la catégorie ou une règle de répartition simple.
- Filtres par statut, priorité, date, demandeur, catégorie et retard.
- Actions en masse limitées aux changements ne présentant pas de risque fonctionnel.

6.7. SLA, échéances et escalades

- Définition d'un délai de prise en charge et d'un délai de résolution par type de demande et priorité.
- Calcul des échéances à partir de la date de soumission ; calendrier ouvert en extension.
- Indicateur visuel : dans le délai, à risque, en retard.
- Notification avant échéance et escalade au responsable en cas de dépassement.
- Suspension éventuelle du compteur lorsque le dossier attend une information du demandeur.

6.8. Notifications

- Centre de notifications dans l'application avec statut lu/non lu.
- Envoi d'e-mails pour les événements importants : soumission, affectation, demande de complément, décision, retard et clôture.
- Liens directs vers la demande concernée.
- Préférences de notification limitées pour éviter la désactivation des alertes obligatoires.
- Exécution asynchrone pour ne pas ralentir les actions utilisateur.

6.9. Tableaux de bord et reporting

- Vue demandeur : demandes en cours, dernières décisions et délais annoncés.
- Vue agent : charge actuelle, éléments en retard et priorités hautes.
- Vue responsable : volumes par statut, catégorie, agent et période.
- Indicateurs : délai moyen de prise en charge, délai moyen de résolution, taux de respect des SLA et taux de réouverture.
- Export CSV des listes filtrées et génération d'un rapport simple en PDF en option.

6.10. Administration et audit

- Gestion des utilisateurs, rôles, équipes, catégories, formulaires, workflows, SLA et modèles d'e-mail.
- Paramètres généraux : formats acceptés, taille maximale des fichiers, durée des sessions et seuils d'alerte.
- Journal d'audit consultable avec filtres par utilisateur, action, objet et période.
- Archivage logique des référentiels utilisés afin de préserver l'historique.
- Page de diagnostic affichant l'état des services techniques sans exposer de secrets.

Cycle de vie standard d'une demande



Des retours vers le demandeur ou l'étape précédente restent possibles selon les règles du processus.

Figure 1 - Exemple de cycle de vie standard d'une demande

7. Règles de gestion

Réf.	Règle
RG-01	Chaque demande possède une référence unique et non réutilisable.
RG-02	Une demande soumise ne peut être supprimée physiquement ; elle peut être annulée ou archivée selon les droits.
RG-03	Le workflow applicable est figé au moment de la soumission afin de garantir la cohérence du dossier.
RG-04	Toute décision de validation ou de rejet doit être associée à un utilisateur habilité et à un horodatage.
RG-05	Le commentaire de rejet est obligatoire. Le commentaire de validation peut être facultatif.
RG-06	Un demandeur ne peut consulter que ses dossiers, sauf rôle complémentaire prévu par l'organisation.
RG-07	Le compteur SLA démarre à la soumission et peut être suspendu uniquement par un statut prévu dans la configuration.
RG-08	Une demande clôturée peut être rouverte pendant une durée paramétrable si le service le permet.
RG-09	Les fichiers exécutables sont refusés. Les extensions, la taille et le type MIME sont contrôlés côté serveur.
RG-10	Les résultats produits par le module IA sont présentés comme une aide et doivent être validés par un utilisateur.
RG-11	Les changements de rôle, de statut, d'affectation et de configuration sont inscrits dans le journal d'audit.
RG-12	Les données archivées restent consultables selon la politique de conservation retenue par l'entreprise.

8. Exigences non fonctionnelles

Domaine	Exigence attendue
Ergonomie	Interface responsive, navigation cohérente, messages d'erreur compréhensibles et limitation du nombre d'actions visibles selon le contexte.
Performance	Pagination des listes, requêtes optimisées, chargement différé des pièces jointes et temps de réponse acceptable sur les opérations courantes.
Disponibilité	Redémarrage simple des services et vérification de santé. Une disponibilité 24/7 n'est pas exigée pour le PFA.
Sécurité	Authentification robuste, contrôle d'accès systématique, validation des entrées, protection contre les attaques web courantes et absence de secret dans le code.
Maintenabilité	Architecture en couches, conventions de code, documentation, tests et faible couplage entre le cœur métier et le module IA.
Compatibilité	Versions récentes de Chrome, Edge et Firefox ; affichage mobile et tablette.
Traçabilité	Logs applicatifs structurés, identifiant de corrélation des requêtes et journal métier séparé des logs techniques.
Confidentialité	Accès aux données selon le besoin métier, masquage des informations sensibles dans les logs et sauvegardes protégées.
Accessibilité	Libellés explicites, navigation clavier sur les écrans principaux, contraste suffisant et messages associés aux champs en erreur.

9. Conception générale

9.1. Architecture proposée

L'application sera organisée autour d'un front-end React consommant une API REST Spring Boot. Les règles métier, les autorisations, le workflow et le calcul des SLA seront gérés par le back-end. PostgreSQL assurera la persistance. Le service IA sera isolé dans un composant Python afin de pouvoir le faire évoluer ou le désactiver sans perturber le fonctionnement principal.

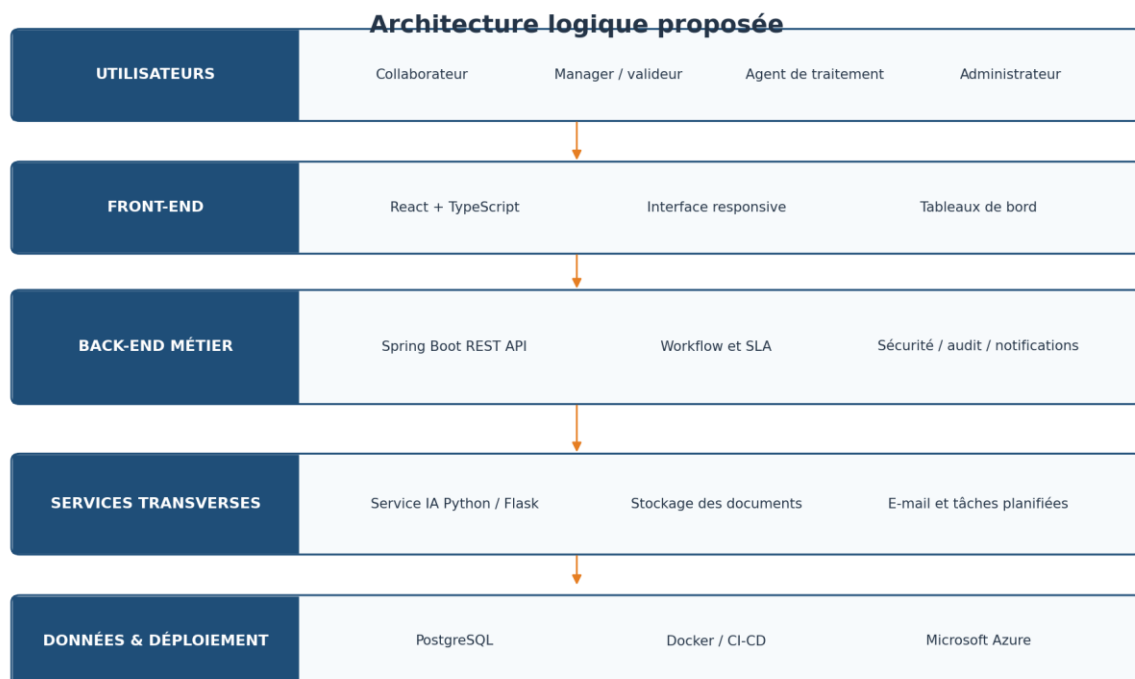


Figure 2 - Architecture logique proposée

9.2. Technologies recommandées

Composant	Technologie	Justification
-----------	-------------	---------------

Composant	Technologie	Justification
Front-end	React, TypeScript, bibliothèque de composants légère	Typage, composants réutilisables et expérience déjà acquise sur cet écosystème.
Back-end	Java 21 ou version LTS disponible, Spring Boot, Spring Security, Spring Data JPA	Socle robuste pour API REST, sécurité et règles métier.
Base de données	PostgreSQL	Transactions, contraintes, JSON si nécessaire et bonnes capacités de reporting.
Service IA	Python, Flask, scikit-learn et LangChain selon le cas	Séparation claire des traitements IA et réutilisation des compétences Python.
Documentation API	OpenAPI / Swagger	Tests rapides et contrat lisible entre front-end et back-end.
Conteneurisation	Docker et Docker Compose	Environnement reproductible pour le développement et la recette.
CI/CD	GitHub Actions ou outil interne équivalent	Compilation, tests, analyse et construction des images.
Cloud	Microsoft Azure : App Service, Container Apps ou VM selon les moyens disponibles	Mise en pratique des notions Cloud et déploiement démontrable.

9.3. Organisation logique du back-end

- Couche API : contrôleurs REST, validation des requêtes et documentation OpenAPI.
- Couche application : cas d'usage, orchestration des transactions et gestion des droits métier.
- Couche domaine : entités, règles de workflow, SLA et invariants.
- Couche infrastructure : accès PostgreSQL, stockage des fichiers, e-mail, tâches planifiées et appels au service IA.
- Couche transverse : journalisation, audit, gestion des erreurs et configuration.

9.4. Écrans principaux à prévoir

Écran	Contenu principal
Connexion	Authentification, récupération de mot de passe, messages de sécurité.
Accueil	Raccourcis, demandes récentes, tâches à traiter et indicateurs adaptés au rôle.
Catalogue	Catégories, recherche, détail d'un service et démarrage d'une demande.
Formulaire de demande	Champs dynamiques, pièces jointes, brouillon et soumission.
Détail d'une demande	Données, statut, actions, commentaires, historique, SLA et pièces jointes.
Mes tâches	File personnelle/équipe, filtres et actions de traitement.
Tableaux de bord	Filtres, cartes KPI, graphiques et listes de retards.
Administration	Référentiels, formulaires, workflows, SLA, utilisateurs, notifications et audit.

10. Modèle de données

Le modèle ci-dessous donne les principales entités. Il pourra être ajusté après l'étude détaillée des processus pilotes. Les noms techniques définitifs seront documentés dans le dossier de conception.

Entité	Rôle dans le système	Relations principales
User	Compte utilisateur et informations de rattachement	Role, Department, Request, TaskAssignment, AuditLog
Role / Permission	Droits fonctionnels	User, périmètre organisationnel
Department / Team	Structure de l'organisation	User, ServiceCatalog, Assignment
ServiceCatalog	Service proposé aux utilisateurs	RequestType, Department, SLA
RequestType	Définition d'un type de demande	FormDefinition, WorkflowDefinition, SLA
FormField	Champ configurable du formulaire	RequestType, FieldOption, RequestFieldValue
Request	Dossier créé par un demandeur	User, RequestType, Status, Comment, Attachment
RequestFieldValue	Valeur saisie pour un champ dynamique	Request, FormField
WorkflowDefinition / Step	Circuit et étapes applicables	RequestType, Transition, Role
RequestHistory	Historique métier horodaté	Request, User, ancien/nouveau statut
TaskAssignment	Responsable actuel ou passé	Request, User/Team, dates
SLA / SLAEvent	Objectifs et événements de délai	RequestType, Priority, Request
Notification	Message destiné à un utilisateur	User, Request, type, état lu

Entité	Rôle dans le système	Relations principales
KnowledgeDocument	Document utilisé pour la recherche assistée	ServiceCatalog, version, indexation
AIAnalysis	Résultat IA et validation humaine	Request, type d'analyse, score, utilisateur
AuditLog	Trace des opérations sensibles	User, action, objet, date, métadonnées

10.1. Contraintes de données importantes

- Clés étrangères et contraintes d'unicité utilisées dès que possible, sans reporter toute la cohérence dans le code.
- Dates stockées dans un format cohérent et affichées selon le fuseau de l'entreprise.
- Suppression logique des référentiels déjà utilisés par une demande.
- Versionnement des définitions de formulaire et de workflow.
- Index sur les colonnes de recherche fréquentes : statut, demandeur, équipe, catégorie, priorité et dates.

11. Interfaces et API

11.1. Principes

- API REST sous le préfixe /api/v1 avec formats JSON documentés.
- Codes HTTP cohérents et format d'erreur commun contenant un code métier, un message et un identifiant de trace.
- Pagination, tri et filtres normalisés sur les listes volumineuses.
- Contrôle d'autorisation dans le back-end pour chaque ressource ; le masquage dans l'interface ne suffit pas.
- Documentation OpenAPI générée et disponible dans l'environnement de développement.

11.2. Exemples de routes

Méthode	Route	Usage
POST	/api/v1/auth/login	Ouvrir une session ou obtenir un jeton selon l'architecture retenue
GET	/api/v1/catalog/services	Lister les services accessibles à l'utilisateur
GET	/api/v1/request-types/{id}/form	Récupérer la définition du formulaire actif
POST	/api/v1/requests	Créer un brouillon
POST	/api/v1/requests/{id}/submit	Soumettre une demande
GET	/api/v1/requests/{id}	Consulter un dossier autorisé
POST	/api/v1/requests/{id}/actions/{actionCode}	Exécuter une transition du workflow
POST	/api/v1/requests/{id}/comments	Ajouter un commentaire
POST	/api/v1/requests/{id}/attachments	Ajouter une pièce jointe contrôlée
GET	/api/v1/tasks	Lister les tâches de l'utilisateur ou de son équipe
GET	/api/v1/dashboards/service	Obtenir les indicateurs d'un service
POST	/api/v1/ai/requests/{id}/analyze	Demander une analyse assistée selon les droits

12. Module d'intelligence artificielle

Le module IA constitue une aide à la qualification et à la recherche. Il ne doit ni approuver une demande, ni la clôturer, ni modifier une donnée sensible sans action explicite d'un utilisateur habilité.

12.1. Fonctions retenues

Fonction	Description	Priorité
Classification	Proposer une catégorie et une priorité à partir du titre et de la description. Le résultat reste modifiable.	P1
Résumé	Produire un résumé court d'une demande longue et de ses derniers échanges.	P1
Recherche documentaire	Répondre à une question en s'appuyant uniquement sur les procédures et FAQ indexées, avec références aux documents.	P2
Suggestion de réponse	Proposer un brouillon de réponse à un agent, sans envoi automatique.	P2
Détection de tendance	Identifier les catégories récurrentes ou les mots-clés fréquents sur une période.	Option

12.2. Approche de réalisation

- Commencer par une base de données d'exemples anonymisés ou générés pour entraîner et évaluer la classification.
- Comparer une approche par règles/mots-clés à un modèle de classification simple. Le choix final devra être justifié par les résultats.
- Pour la recherche documentaire, découper les documents, calculer des représentations vectorielles et retourner les passages utilisés comme sources.
- Stocker le résultat, le niveau de confiance et la correction éventuelle de l'utilisateur pour permettre une évaluation.
- Prévoir un mode désactivé : l'application principale doit rester utilisable sans le service IA.

12.3. Critères d'évaluation du module IA

Critère	Mesure proposée
Classification	Précision, rappel, F1-score et matrice de confusion sur un jeu de test séparé
Résumé	Évaluation humaine sur la fidélité, la concision et l'absence d'information inventée
Recherche documentaire	Taux de réponses appuyées par un passage pertinent et présence des références
Utilité	Temps gagné estimé et taux d'acceptation/correction des suggestions

Point de vigilance - Aucune donnée réelle confidentielle ne doit être envoyée vers un service externe sans validation de l'entreprise. Un modèle local ou des données anonymisées seront privilégiés pour la démonstration.

13. Sécurité et traçabilité

Risque	Mesure attendue
Accès non autorisé	Spring Security, contrôle de rôle et de périmètre sur chaque opération.
Vol de mot de passe	Hachage robuste, politique minimale, limitation des tentatives et réinitialisation sécurisée.
Injection / données invalides	Requêtes paramétrées, validation Bean Validation, contrôle des identifiants et filtrage des entrées.
XSS / CSRF	Échappement côté interface, en-têtes de sécurité, stratégie adaptée au mode d'authentification.
Fichiers dangereux	Liste blanche d'extensions, contrôle MIME, renommage interne, stockage hors répertoire public et analyse antivirus si disponible.
Fuite dans les logs	Aucun mot de passe, jeton ou contenu sensible ; masquage des données personnelles.
Élévation de privilèges	Tests d'autorisation, séparation administration fonctionnelle/technique et journalisation des changements.
Perte de données	Sauvegarde PostgreSQL, procédure de restauration testée et conservation des fichiers cohérente avec la base.
Usage IA non maîtrisé	Validation humaine, journal des appels, seuil de confiance et possibilité de désactivation.

13.1. Journal d'audit

Le journal d'audit devra au minimum contenir l'utilisateur, la date, l'action, le type d'objet, l'identifiant concerné, le résultat et un résumé des changements. Les données d'audit seront consultables en lecture seule et protégées contre la modification depuis l'interface.

14. Stratégie de tests

Niveau	Contenu	Outils possibles
Tests unitaires	Règles métier, calcul SLA, transitions, validations et services	JUnit 5, Mockito, pytest
Tests d'intégration	Repositories, sécurité, transactions et API avec base de test	Spring Boot Test, Testcontainers
Tests front-end	Composants critiques, formulaires et validations	Vitest/Jest, React Testing Library

Niveau	Contenu	Outils possibles
Tests API	Scénarios nominaux, erreurs, droits et pagination	Postman/Newman ou tests automatisés
Tests fonctionnels	Parcours complets par rôle sur les processus pilotes	Cahier de recette
Tests de sécurité	Accès horizontal/vertical, fichiers, entrées invalides et dépendances	OWASP ZAP en contrôle de base, analyse des dépendances
Tests de performance	Listes, tableau de bord et soumission sous charge raisonnable	JMeter ou k6 en option

14.1. Scénarios de recette obligatoires

- Création d'une demande valide, sauvegarde en brouillon puis soumission.
- Refus d'une soumission avec champ obligatoire manquant.
- Validation puis affectation à une équipe et à un agent.
- Retour au demandeur pour complément puis reprise du traitement.
- Dépassement d'un SLA avec notification et escalade.
- Tentative d'accès à une demande d'un autre service par un utilisateur non autorisé.
- Ajout d'un fichier interdit ou dépassant la taille maximale.
- Modification d'un workflow publié sans impact sur une demande déjà en cours.
- Utilisation du module IA puis correction manuelle de sa proposition.
- Clôture, réouverture autorisée et vérification de l'historique.

15. Déploiement et DevOps

15.1. Environnements

Environnement	Objectif	Données
Développement	Travail local du stagiaire et tests rapides	Données fictives
Intégration	Validation automatique des builds et tests	Jeu de test reproductible
Recette / démonstration	Validation fonctionnelle avec l'encadrant	Données anonymisées ou de démonstration

15.2. Pipeline minimal

1. Vérification du format et compilation du front-end et du back-end.
2. Exécution des tests unitaires et d'intégration.
3. Analyse des dépendances et rapport de qualité de code si l'outil est disponible.
4. Construction des images Docker avec version issue du tag Git.
5. Déploiement manuel contrôlé en recette, puis automatisation possible après stabilisation.
6. Conservation des variables sensibles dans les secrets du pipeline ou du cloud.

15.3. Exploitation minimale

- Endpoint de santé pour le back-end, la base et le service IA.
- Logs centralisables et niveaux de log configurables.
- Script documenté de sauvegarde et de restauration PostgreSQL.
- Procédure de démarrage, arrêt, mise à jour et retour à la version précédente.

16. Planning prévisionnel

Le projet est organisé sur 12 semaines. Le planning conserve les phases de cadrage, de conception, de réalisation, de test, de déploiement et de documentation.

Période	Travaux principaux	Résultat attendu
S1	Cadrage, choix des processus pilotes et analyse de l'existant	Note de cadrage et périmètre validé
S2	Spécifications, règles de gestion, maquettes, modèle de données et architecture	Dossier de conception validé

Période	Travaux principaux	Résultat attendu
S3	Initialisation Git, socles front-end/back-end, PostgreSQL, Docker et CI	Socle technique opérationnel
S4	Authentification, profils, organisation, rôles et droits	Gestion des utilisateurs sécurisée
S5	Catalogue, formulaires configurables, brouillon et soumission	Premier parcours demandeur complet
S6	Workflow, validations, historique et files de travail	Premier processus pilote fonctionnel
S7	Affectation, commentaires, pièces jointes et notifications	Circuit de traitement complet
S8	SLA, échéances, escalades, tableaux de bord et exports	Pilotage opérationnel disponible
S9	Classification et résumé assistés par IA, avec évaluation des résultats	Fonctions IA évaluées et intégrées
S10	Sécurité, tests front-end/API, recette interne et correction des anomalies	Version candidate à la recette
S11	Déploiement, sauvegarde, documentation d'exploitation et recette	Plateforme déployée et PV de recette
S12	Ajustements finaux, rapport, support de maintenance et démonstration	Livrables finaux

16.1. Jalons de validation

Jalon	Échéance	Condition de validation
J1 - Cadrage	Fin S1	Processus pilotes, acteurs et périmètre approuvés
J2 - Conception	Fin S2	Maquettes, modèle de données et architecture validés
J3 - MVP	Fin S6	Soumission et workflow complet sur un processus pilote
J4 - Version fonctionnelle	Fin S8	SLA, notifications et tableaux de bord disponibles
J5 - Version candidate	Fin S10	Tests critiques passés et anomalies bloquantes corrigées
J6 - Livraison	Fin S12	Recette, déploiement, documentation et démonstration terminés

17. Livrables et critères de recette

17.1. Livrables attendus

- Note de cadrage et cahier des charges validé.
- Dossier de conception : architecture, diagrammes UML, modèle de données et règles de sécurité.
- Maquettes ou prototype des écrans principaux.
- Code source front-end, back-end et service IA dans un dépôt Git propre.
- Scripts de création/migration de la base de données et jeu de données de démonstration.
- Images Docker, fichier Docker Compose et pipeline CI/CD.
- Documentation OpenAPI et guide d'installation.
- Plan de tests, résultats et procès-verbal de recette.
- Guide utilisateur court pour les principaux rôles.
- Rapport PFA, support de maintenance et scénario de démonstration.

17.2. Critères de recette fonctionnelle

ID	Critère	Résultat attendu
REC-01	Connexion et droits	Chaque rôle accède uniquement aux fonctions et données prévues.
REC-02	Catalogue et formulaire	Un utilisateur peut sélectionner un service, compléter les champs et soumettre une demande valide.
REC-03	Workflow	La demande suit les étapes configurées et seules les actions autorisées sont proposées.
REC-04	Historique	Toutes les actions importantes sont visibles avec auteur, date et contenu utile.
REC-05	Affectation	Un agent ou une équipe reçoit la tâche et la retrouve dans sa file.
REC-06	SLA	Les échéances, alertes et retards sont calculés selon la configuration.
REC-07	Notifications	Les principaux événements génèrent une notification exploitable.
REC-08	Tableaux de bord	Les indicateurs correspondent aux données de test et respectent les filtres.
REC-09	Pièces jointes	Les fichiers autorisés sont accessibles aux utilisateurs habilités et les fichiers interdits sont refusés.
REC-10	IA	La proposition est présentée comme une aide, traçable et corrigable.
REC-11	Déploiement	La procédure documentée permet de démarrer l'application dans l'environnement cible.

17.3. Démonstration finale recommandée

1. Connexion comme collaborateur et création d'une demande depuis le catalogue.
2. Validation par un manager puis affectation à une équipe.
3. Qualification assistée par IA, avec correction de la catégorie proposée.
4. Traitement, ajout d'un commentaire et d'une pièce jointe.
5. Affichage du délai, simulation d'une alerte et consultation du tableau de bord.
6. Clôture puis consultation de l'historique et du journal d'audit.

18. Risques et points de vigilance

Risque	Impact	Mesure de réduction
Périmètre trop large	Retard et fonctionnalités incomplètes	Prioriser le MVP et limiter à deux processus pilotes.
Processus métier non validé	Reprise de conception	Organiser des validations courtes avec l'encadrant à chaque jalon.
Données insuffisantes pour le ML	Modèle peu fiable	Prévoir une approche par règles et un jeu de données anonymisé/généré.
Dépendance à un service IA externe	Coût, confidentialité ou indisponibilité	Isoler le service, prévoir un mode local ou désactivé.
Complexité des formulaires dynamiques	Dérive technique	Limiter les types de champs et les conditions dans la version PFA.
Faibles d'autorisation	Exposition de données	Tests systématiques par rôle et contrôle côté back-end.
Déploiement tardif	Problèmes découverts en fin de projet	Déployer une première version technique dès les premières semaines.
Documentation reportée	Livraison difficile à reprendre	Mettre à jour la documentation à chaque sprint.

19. Organisation du suivi

Un point hebdomadaire de 30 à 45 minutes est recommandé avec l'encadrant. Le stagiaire présentera les éléments terminés, les difficultés rencontrées, les décisions à prendre et le plan de la semaine suivante. Les demandes importantes devront être inscrites dans un outil de suivi afin d'éviter les changements informels de périmètre.

Rythme	Support	Contenu
Hebdomadaire	Compte rendu court	Avancement, blocages, décisions et prochaines tâches
Toutes les 2 semaines	Démonstration	Fonctions réellement utilisables sur l'environnement de test
À chaque jalon	Validation formelle	Périmètre, conception, MVP, recette et livraison

20. Conclusion

SmartFlow IT constitue un sujet de PFA équilibré entre analyse métier, gouvernance, développement logiciel, sécurité, Cloud et intelligence artificielle. Le résultat attendu n'est pas seulement une démonstration technique : il s'agit d'une application cohérente, documentée et déployable, construite autour de processus réels et d'indicateurs utiles.

Les deux processus pilotes, les règles de SLA et le mode d'hébergement seront arrêtés durant la première semaine de cadrage avec l'entreprise d'accueil. Toute extension sera engagée uniquement après validation du socle fonctionnel.